

Course: Special Ship Automation Systems 2026
Homework Assignment (HW)
System Modeling Using Experimental
Measurement Data (Part 2)

Instructor: Prof. Georgios Papalambrou

May 2026

1 Overview

This is the second and final part of the homework assignment. Building on the data analysis, preprocessing, and simple neural network models you developed in Part 1, you will now improve your models through feature engineering, deeper architectures, and systematic hyperparameter optimization.

2 Tasks to Solve

For each one of the two parameters chosen in **Part 1**, you will need to improve their models by doing the following:

1. Create new input features by combining the original variables (**feature engineering**). Justify the criteria used to select or construct the new features (physical reasoning, correlation analysis, domain knowledge, etc.).
2. Develop **deeper or more complex networks** to improve model fitting and generalization over the simple networks from Part 1. Justify your architectural choices.
3. Use **Optuna** [1] or **Weights & Biases (WandB)** [2] for hyperparameter tuning and experiment tracking. Alternatively, create a small hyperparameter grid manually and track the training process in an **appropriate folder structure**.
4. Evaluate the results on the test set and provide plots of the training progress (Training loss vs. iteration, Validation loss vs. iteration). Compare the performance of your improved models against the simple networks from Part 1.
5. For each change you introduce (e.g. new features, additional neurons, extra layers, different activation functions, regularization), **present how it affects the training process and the final performance**. For every change, either plot the full training history or provide an indicative trend

showing the effect of that change (e.g. more neurons $\rightarrow$ lower error, more layers $\rightarrow$ lower error initially but higher error beyond a certain threshold). Additionally, provide a **summary table** listing each change alongside its corresponding total (test) loss, so the effect of every modification is clearly quantifiable. **Comment on and explain** the observed behaviors.

6. Compute and present **SHAP (SHapley Additive exPlanations) values** for your best-performing models. Produce summary plots showing the feature importance ranking and discuss whether the resulting importances are consistent with the expected physical behavior of the system. If you introduced engineered features, comment on whether they contribute meaningfully according to SHAP. A minimal code example is provided below for reference.

3 SHAP Code Example

The following snippet demonstrates how to compute and visualize SHAP values for a trained PyTorch neural network. Adapt it to your own model and data.

```
"""
SHAP (SHapley Additive exPlanations) Example for Neural Network Models
=====
Course: Special Ship Automation Systems 2026
Homework Assignment - Part 2

This script demonstrates how to compute and visualize SHAP values
for a trained neural network. Adapt it to your own model,
data, and feature names.

Requirements:
    pip install shap torch matplotlib numpy

Reference:
    Lundberg, S. M. & Lee, S.-I. (2017). A Unified Approach to
    Interpreting Model Predictions. NeurIPS 30, pp. 4765-4774.
    https://github.com/shap/shap
"""

import matplotlib.pyplot as plt
import numpy as np
import shap
import torch

# Load you model class here
# from your_model_module import YourModel

# Load your trained model and data here
np.random.seed(42)
n_features = ...
feature_names = ...
X_train = np.random.randn(500, n_features).astype(np.float32)
X_test = np.random.randn(100, n_features).astype(np.float32)

# Instantiate model and load weights
# model = YourModel(...) # replace with your model instantiation
model = ... # replace with your model instantiation
# model.load_state_dict(torch.load("best_model.pt"))
model.eval()

# Create a prediction function for SHAP
# Uncomment ONE of the following predict functions depending on your model type (PyTorch o

# For PyTorch models:
# def predict(x):
```

```

#     """Wrapper so SHAP can call the model with numpy arrays."""
#     with torch.no_grad():
#         tensor = torch.tensor(x, dtype=torch.float32)
#         output = model(tensor).numpy()
#         return output.flatten()

# For TensorFlow/Keras models:
# def predict(x):
#     """Wrapper so SHAP can call the model with numpy arrays."""
#     return model.predict(x, verbose=0).flatten()

# Use KernelExplainer which is model-agnostic but requires a background dataset,
# Lower n_background for faster computation, while more for better estimates.
n_background = 1_000
idx = np.random.choice(X_train.shape[0], n_background, replace=False)
background = X_train[idx]

# Instantiate the SHAP explainer and compute SHAP values for the test set
explainer = shap.KernelExplainer(predict, background)

# Compute SHAP values for the test set
shap_values = explainer.shap_values(X_test)

# Update plotting style according to h/w description.
# 6. Visualize: Summary bar plot (mean |SHAP| per feature)
shap.summary_plot(
    shap_values,
    X_test,
    feature_names=feature_names,
    plot_type="bar",
    show=False,
)
plt.title("SHAP Feature Importance (mean |SHAP value|)")
plt.tight_layout()
plt.savefig("shap_bar.png", dpi=150, bbox_inches="tight")
plt.show()

# 7. Visualize: Beeswarm plot (feature value vs. SHAP impact)
shap.summary_plot(
    shap_values,
    X_test,
    feature_names=feature_names,
    show=False,
)
plt.title("SHAP Beeswarm Plot")
plt.tight_layout()
plt.savefig("shap_beeswarm.png", dpi=150, bbox_inches="tight")
plt.show()

```


4 Notes:

1. The data for the assignment can be found on the course website on Helios.
2. For step 2, you are free to choose the network architecture (number of layers, neurons, activation functions, regularization, etc.). You may use ideas from the literature (cite sources if applicable).
3. All plots must include **axis labels, curve labels, grids, and must not be screenshots** (use tips from the following repo *BeautifulFigures* as well as the videos in the author's channel *Principles of Beautiful Figures for Research Papers*).
4. Code must be in **Python**.
5. Code must include comments explaining the process.
6. Using Git and WandB will be taken positively into consideration.
7. Presentations must be **20 slides** (covering both parts combined). Each team will have **~20 minutes** for the presentation, followed by **~5 follow-up questions. In-person attendance is mandatory.**
8. For the presentation, use the template provided in Helios and follow the rules from the **How to present.pdf** in the extra material folder, as well as the general ideas from the *How to Speak* video.
9. The examination date is **12/6/2026**.

References

- [1] T. Akiba et al., "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2019, pp. 2623–2631. DOI: 10.1145/3292500.3330701.
- [2] L. Biewald, "Experiment Tracking with Weights and Biases," 2020. [Software]. Available: wandb.ai.
- [3] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774. Available: github.com/shap/shap.